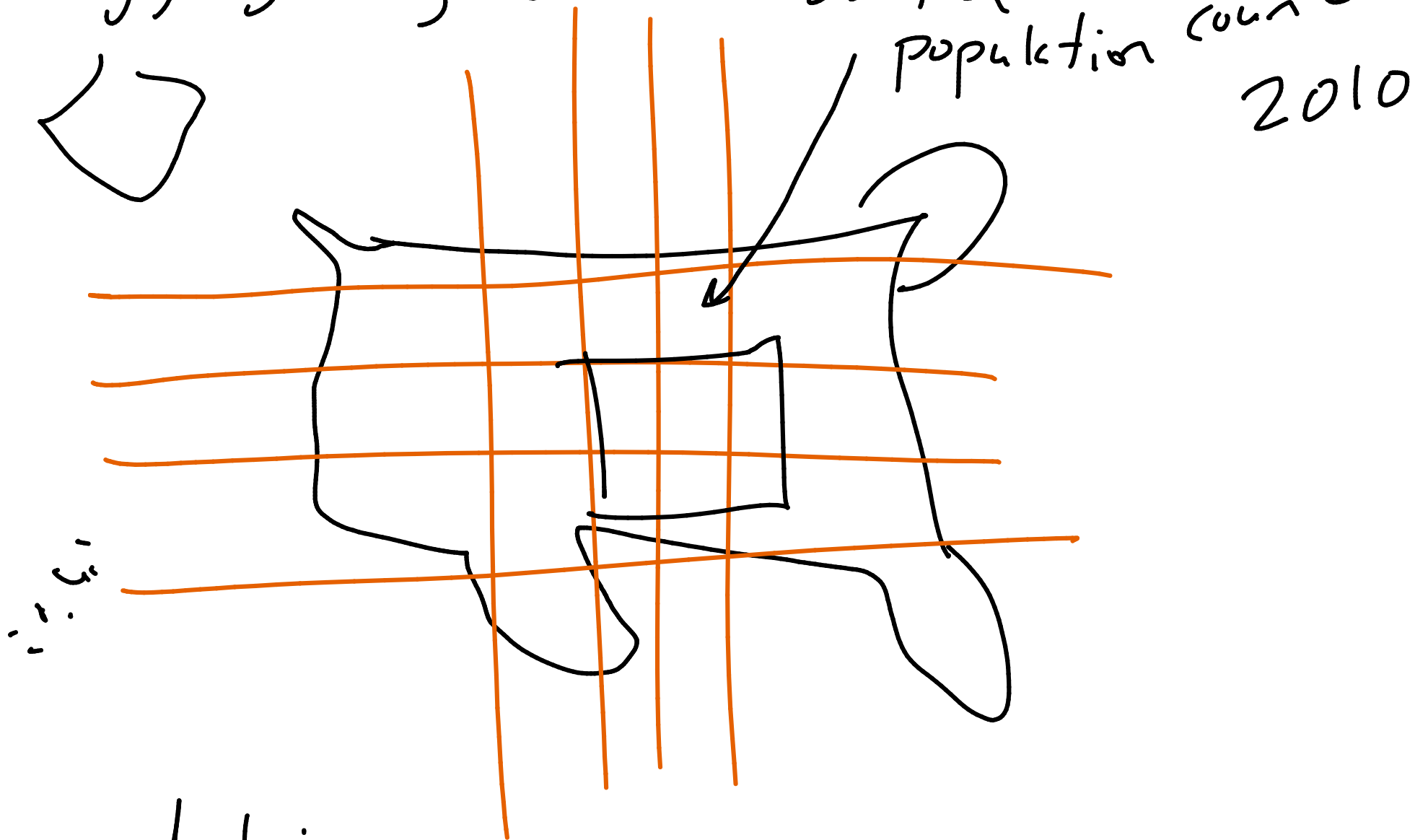


Today: shared memory concurrency
and all the bad/and good aspects
[How can we reasonably code?]

Also: new census assignment
(working w/ partners, etc)

Aggregating census data



data:

<u>latitude</u>	<u>longitude</u>	<u>population</u>
37.9	-180.6	2387

Write code to answer queries

"how many people in

latitude range $[x_1, x_2)$

longitude range $[y_1, y_2)$

preprocessing: - find min max lat, long

- arrange for area to be split into
a given # of bins

Preprocess(30, 40, ...)

↑
30 bins on the x-axis

← 40 bins on the y-axis

i.e., how close those orange lines
should be

query(10, 15, 20, 35)

how many in x bins ✓ not inclusive
people 10 → 15 | y bins

20 → 35

5 parts

First part is due ~~Friday~~
Monday

Second Monday

Third Wed

...

First 2 parts are partners

Part 5, in addition to code has
experimental questions

... what cutoff is best? ...

Each part (1 $\rightarrow$ 5) is broken into
2 pieces

	Coding	1 analysis question	
1	3 pts	1 pt	$\rightarrow$ one 4 pt assignment
2			
3			
4			
5			

The pitfalls of shared memory
• we already know that multiple threads accessing a variable that might change is a problem.

```
> javap -c Sample1Kt
Compiled from "Sample1.kt"
public final class Sample1Kt {
    public static final void main();
```

Code:

```
0: sipush          500
3: istore_0
4: bipush          100
6: istore_1
7: iload_0
8: iload_1
9: isub
10: istore_0
11: getstatic       #12
```

```
a/io/PrintStream;
```

```
14: iload_0
15: invokevirtual #18
```

```
tln:(I)V
```

```
18: return
```

```
public static void main(java.lang.String[])
```

Code:

```
0: invokestatic   #24
3: return
```

```
}
```

push 500 onto stack
pop off stack, store in
var 0
← push 100

← pop and store
gets vars 0 and 1 back
again
← sub

← store

Imagine we have a function to withdraw from a bank account

```
fun withdraw(amount) {  
    b = balance  
    a = amount  
    ans = b - a  
    balance = ans  
}
```

shared variable

this is less detailed than the bytecode

Imagine 2 threads acting at same time

Schedule

Thread 1

b = balance

a = amount

Thread 2

b = balance

a = amount

ans = b - a

balance = ans

ans = b - a

balance = ans

We tried doing 2 withdrawals
(only one actually happens)

BAD

— Is this high level sufficient?

- if this is bad, reality is worse

Can we have 2 ops happen at
precisely the same time?

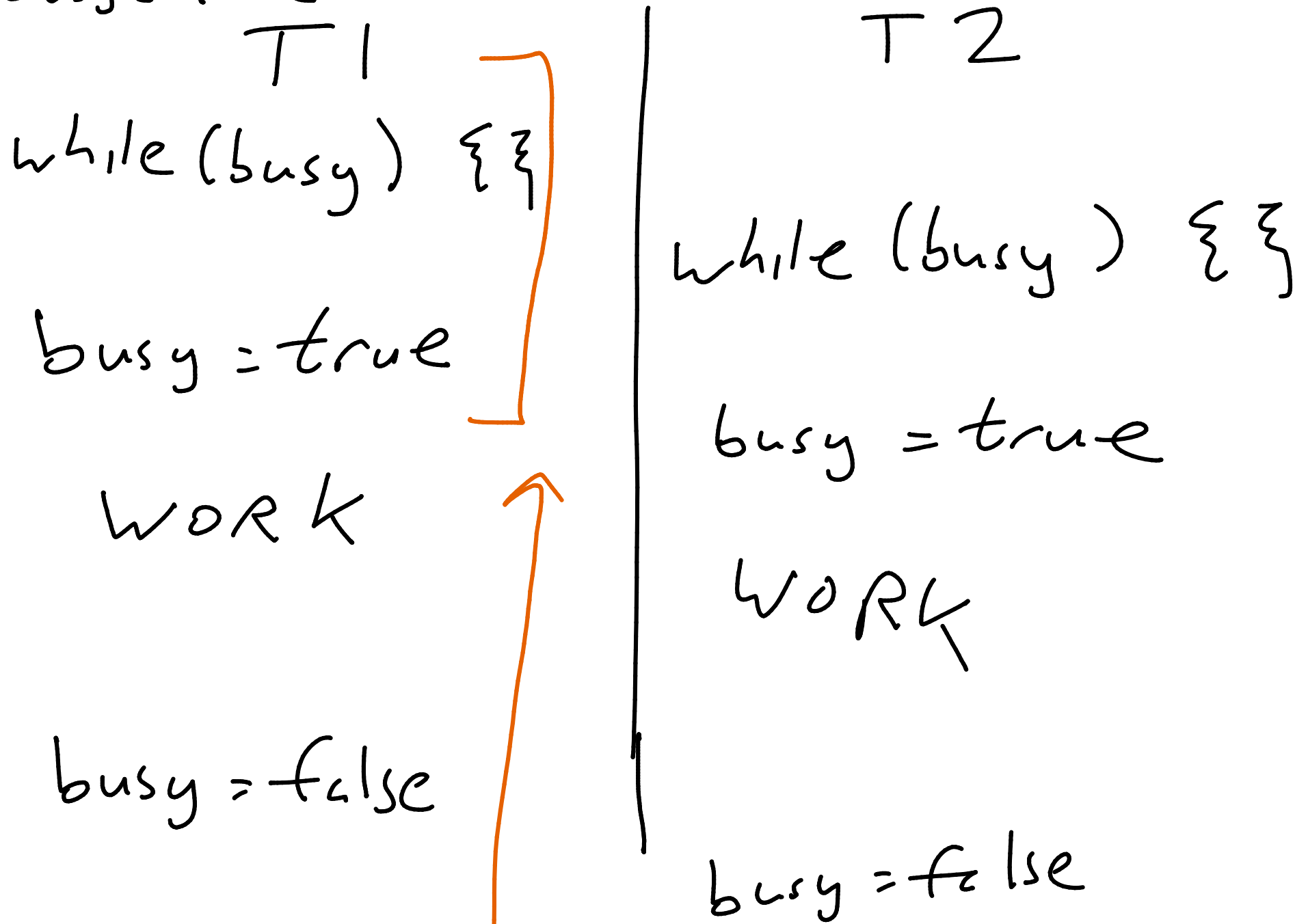
(all the schedules we write
will interleave, but nothing will
happen in more than one thread
on the same line)

- I don't know(??) ugly
- can we start something when
something else is $\frac{1}{2}$ done? Ugh

Pretend that each line of

low level code happens not at
"exactly same time" as something
else. We can set by who that

Reading talked about a bad strategy
that looked like
 $busy = false$



If I can use a lock, the OS/machine
language enforces nothing else can get in between

Lots of ways of locking:

"synchronized" on an object

You can use any object as something
"to hold" to prevent others
from accessing a section of code.

↳ works in threads, not in
coroutines